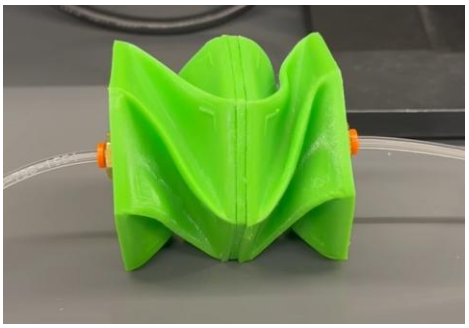
Embedded Control System for Soft Robotic Crawler



Author: Ojasvi Mukkoti

Advisor: Professor Kris Dorsey

PARSEs Research Lab

Northeastern University

April 22, 2026

Abstract

In this project, a design and implementation of an embedded control system for a soft robotic Crawler was done. The Crawler actuates with two pneumatic Kresling structures, which are popular in the soft robotic research environment. The system uses an ESP32 microcontroller to control air pumps and solenoid valves through MOSFET drivers, enabling controlled inflation and deflation of the two Kreslings. Hardware architecture was developed to support actuation, while embedded C++ software was implemented to generate timed control sequences for crawling motion. Wireless communication was integrated using micro-ROS over Wi-Fi, which allows remote command execution and system testing. Experimental validation demonstrated successful control and coordinated crawling behavior. The results show the feasibility of combining embedded system and soft robotics for locomotion and provide a foundation for future improvements in mechanical leg designs, control strategies, and system integration.

1. Introduction

Soft robotics is a subfield of robotics that focuses on machines that are built from flexible and deformable materials that can mimic biological structures. These can be materials such as silicon or rubbers that can bend, twist, and squeeze. Unlike rigid robotics that use rigid motors, soft robotics allow alternative actuation methods such as inflatable chambers [7]. This allows for foldable robotics that can crease into specific patterns to create 3D structures to assist with specific movements. This type of soft robotics is called origami, where these robots are a hybrid between rigid and soft robotics. They leverage the principles of origami, which is an ancient art form of folding flat materials into 3D structures. Origami robots' frameworks allow them to withstand a greater mechanical load without compromising the shape or stability of the structure.

In this project, the specific type of Origami pattern used is the Kresling. The Kresling pattern is one that generates rotational and contractional movement under compressive force, which helps with bioinspired design such as the Crawler [6]. A soft robotic Crawler is a mechanism that mimics the motion of caterpillars or earthworms, which allows the machine to navigate properly in complex or confided spaces. The Crawler used in the PARSEs Lab at Northeastern University has the dimensions of roughly 3.5 x 2.25 in, which allows for applications where manipulation of objects is needed.

Existing soft robotic systems often rely on tether control setups, external pneumatic sources, or manual valve operation, which limit mobility and real-time adaptability. Additionally, many systems use centralized control architectures without integrated wireless communication, restricting their use in autonomous or distributed applications [1]. In contrast, this project develops a compact, embedded control system that integrates pneumatic actuation, wireless communication, and real-time control on a single platform. By leveraging an ESP32 microcontroller and micro-ROS over Wi-Fi, the system enables remote user control and coordinated actuation without the need for external control infrastructure.

The goal of this project is to design and implement an embedded system capable of controlling the crawler's movement through wireless commands. This requires integrating multiple subsystems – including pumps, valves, communication modules, and power regulation – into a reliable and scalable platform. Through this work, a fully functional soft robotic system is developed from the ground up, demonstrating the feasibility of embedded, wireless control for pneumatic soft robots.

II. Background

Soft robotics is a field of robotics that focuses on designing systems constructed from tractable materials, enabling flexible and adaptive motion. These materials can deform continuously, making them well-suited for applications in unstructured or confined environments. Actuation in soft robotics is commonly achieved through pneumatic systems, where pressurized air is used to inflate or deflate the structure. In this project, actuation is achieved by using Kresling structures, an origami-inspired geometric shape that exhibits coupled rotational and axial contraction behavior under pressure changes. This property makes Kreslings well-suited for bio-inspired motion mechanisms such as crawling.

Embedded control systems play a critical role in allowing precise actuation of soft robotic systems. These systems typically consist of a microcontroller interfacing with sensors and actuators to execute control logic. There are two primary control models that are commonly used: open-loop and closed-loop control. Open-loop control is a system that operates without feedback. This means that the system's output does not get adjusted based on its current state. In contrast, closed-loop control incorporates feedback from sensors to continuously compare the system's output to a desired state and adjust accordingly, which improves accuracy and robustness [2][8]. This distinction is widely studied in robots and automation, where closed-loop systems are often preferred for tasks requiring high precision, while open-loop systems are sufficient for predictable, repeatable operations [3]. Additionally, prior work comparing open and closed loop control in automated machinery highlights how feedback-based control improves performance in dynamic environments [4]. The system developed in this project operates in an open-loop manner, where predefined timing sequences control the inflation and deflation of the two Kreslings.

Commented [DK1]: PF: Cite 1-2 papers here to illustrate this contrast and give links to related work.

The soft robotic Crawler is a class of bioinspired robot designed to replicate the locomotion of organisms such as earthworms or caterpillars [5]. These systems achieved movement through cyclic deformation, where alternating expansion and contraction of the pieces results in forward motion. One common approach uses origami-inspired structures, like the Kresling pattern, which enables coupled rotational and axial deformation under actuation. By integrating multiple Kresling modules in series and controlling their inflation and deflation, these robots can generate directional movement. Pneumatic actuation is usually used, where air pumps and valves regulate internal pressure to drive deformation, making these robots well-suited for motion in constrained or unstructured environments.

Commented [DK2]: PF: Cite

III. System Architecture

The embedded control system for the soft robotic crawler was designed to enable controlled inflation and deflation of two Kresling actuators through electrically controlled pneumatic components. Each Kresling requires one air pump for inflation and one solenoid valve for controlled deflation, resulting in an architecture that can be replicated across both actuators. A simple block diagram of the hardware connections for one Kresling is shown in Fig. 1.

Commented [DK3]: PF: This is a good place for a figure and reference

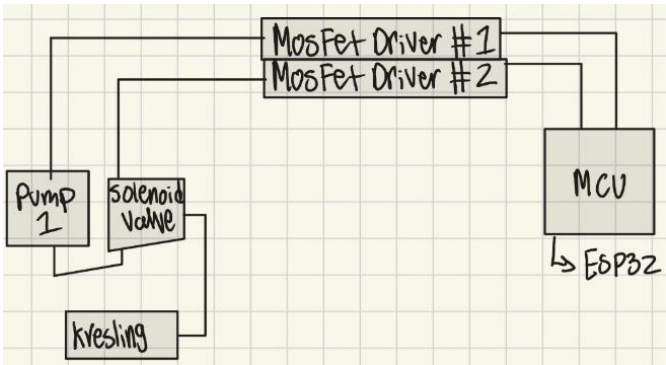


Fig. 1: Block Diagram for one Kresling.

At the core of the system is the Adafruit ESP32 HUZZAH Feather microcontroller unit (MCU). This device was selected due to its integrated Wi-Fi capability, which supports wireless communication with an external host system, as well as its sufficient number of general-purpose input/output (GPIO) pins required to interface with multiple actuators and driver circuits. The MCU operates at 3.3 V logic levels and is responsible for generating control signals based on incoming commands.

A. Signal Flow and Operation

The system follows a hierarchical signal flow from high-level control inputs to physical actuation, which is shown in Fig. 2. The control commands are transmitted wirelessly from a host computer to the ESP32. The commands are transmitted with a mirco-ROS script that runs on the host computer. Upon receiving these commands, the MCU processes the desired actuation sequence and outputs corresponding digital signals to its GPIO pins. These signals are then routed to MOSFET-based drivers, which function as switching interfaces between the low-power control domain and higher-power actuation components.

Based on the user's command, the corresponding GPIO pins will be enabled to be high or low, which will turn specific MOSFET drivers on or off. More about the software structure will be explained in section IV. Once the corresponding MOSFET is activated, this allows current to flow through the connected air pump or solenoid valve. This results in inflation or deflation of

the Kresling actuator, producing the desired deformation and contributing to the Crawlers' movement. As shown in Fig. 3, alternating actuation between the two modules generates forward movement: Fig. 3A illustrates the deflation of one Kresling, while Fig. 3B shows the subsequent deflation of the second module, together forming a coordinated crawling sequence.

Commented [DK4]: PF: Include image or video as figure

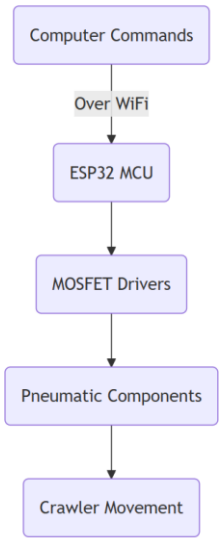


Fig. 2: High-Level Signal Flow Chart.

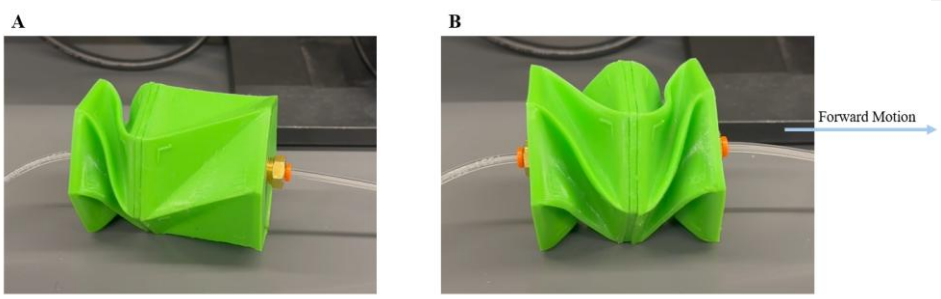


Fig. 3: Actuation of Crawler

B. MOSFET Driver Design

MOSFET drivers were implemented as a necessary interface between the ESP32 and the pneumatic components, which operate at significantly higher current and voltage levels than the

microcontroller can safely supply. The GPIO pins of the ESP32 are limited in current sourcing capability and cannot directly drive the air pumps and solenoid valves, without risking damage to the MCU or unstable operation. Therefore, an intermediate switching stage is necessary to bridge the low-power control signals and high-power actuation requirements.

In this project, the Adafruit MOSFET drivers were used for each pneumatic component: air pumps and solenoid valves. Each Kresling component is controlled using an N-channel MOSFET configured as a low-side switch. In this configuration, the power pins, V+ and ground, are connected to their corresponding terminals of positive and ground on the breadboard, while the input signal pin of the MOSFET was connected to an ESP32 GPIO pin. The MOSFET driver also has output terminal blocks which are connected to the positive and negative signals of the pneumatic components. A full schematic of all the hardware connections for the control system is in Fig. 4.

When the gate voltage exceeds the threshold voltage, the MOSFET enters conduction, enabling current flow through the load and activating the component. This design is necessary not only to enable actuation, but also to ensure system reliability and safety. It isolates the low-power control circuitry from high-power components, protects the microcontroller from overcurrent conditions, and enables efficient switching with minimal power loss.

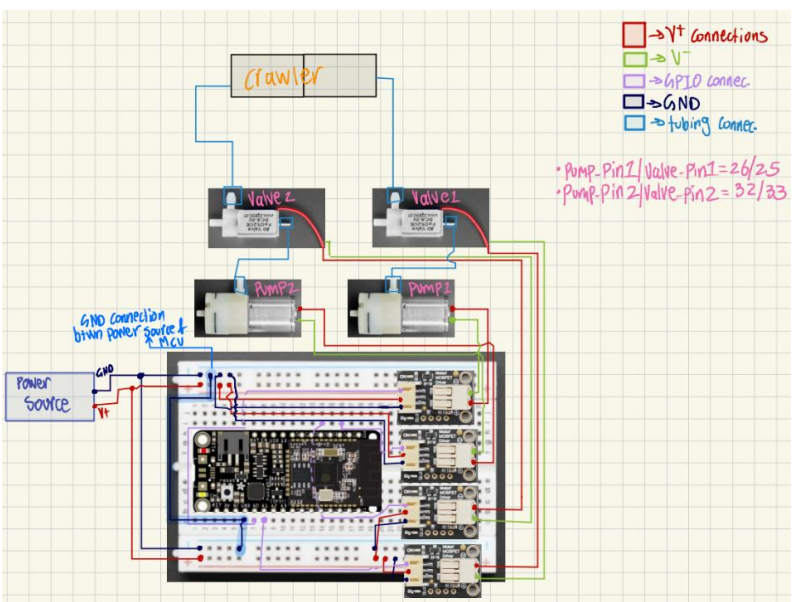


Fig. 4: Schematic of Full Embedded Control System for Crawler.

IV. Embedded System Design

The embedded system was designed to control pneumatic components, air pumps and solenoid valves, for each Kresling of the Crawler. This was done by selectively activating air pumps and solenoid valves which were each attached to a MOSFET driver. The control logic determines which components are enabled at a given time to achieve either inflation or deflation of the Kresling.

A. Hardware Interconnection and Pneumatic Pathway

Each Kresling is attached to an air pump which in turn is attached to a solenoid valve. This valve is used to regulate air flow into and out of the Kresling. The air pump is responsible for generating airflow, while the valve controls whether air is retained within or released from the Kresling. Electrically, both the pump and valve are connected to the ESP32 through individual MOSFET drivers. The MOSFETs act as low-side switches, allowing the MCU to control high-current components through low-power GPIO signals.

Commented [DK5]: PF: I'm not sure what this means

From a pneumatic perspective:

- The air pump outlet is connected to a Kresling chamber.
- The solenoid valve is placed along the airflow path to control release of air.
- When the valve is closed, air is retained within the structure.
- When the valve is opened, air is allowed to escape, allowing for deflation.

This configuration allows independent control over airflow generation and on release, which is critical for achieving controlled deformation.

B. MOSFET-Controlled Actuation Logic

The system relies on coordinated switching of MOSFET-controlled components to achieve two primary modes of operation: inflation and deflation.

Each Kresling is controlled using two GPIO signals:

- One for the pump MOSFET.
- One for the valve MOSFET.

The control logic in the embedded software determines the ON/OFF states of these MOSFETs.

1.) Inflation (Air Intake)

To inflate the Kresling module, the valve and pump MOSFETs were turned off. This allows air to flow back into the Kresling and therefore inflate the device back to its original form. In code implementation the digitalWrite command was used with the valve/pump pin indicated and whether that specific pin would be low or high – on or off.

2.) Deflation (Air Release)

To deflate the Kresling, the valve MOSFET is off while the pump MOSFET is on. In this state, airflow generation is stopped, and the system allows air within the Kresling to leave. This reduces internal pressure, causing the Kresling to contract.

In practice, testing revealed that passive deflation (simply stopping the pump) was insufficient. Therefore, coordinated valve and pump movements were necessary to enable air removal and deflation of the module.

C. Dual Kresling Coordination

The system extends the single-actuator control architecture to two independent Kresling modules, each consisting of a dedicated air pump and solenoid valve controlled through separate MOSFET driver circuits. This modular design enables independent and coordinated actuation of each segment, which is essential for generating crawling movement.

At the software level, high-level user commands are mapped to specific actuator states, allowing direct control over the inflation and deflation of each Kresling:

- **I / A**: Inflate Kresling 1 / 2
- **D / E**: Deflate Kresling 1 / 2
- **C**: Execute a coordinated crawling sequence
- **O**: Disable all actuators

The crawling motion is achieved through a time-sequenced actuation pattern that alternates between the two Kreslings. Specifically, one actuator is inflated while the other is deflated, creating asymmetric deformation along the body of the crawler. This asymmetry generates differential friction with the ground, resulting in forward displacement.

To ensure effective locomotion, the control sequence incorporates carefully tuned timing delays between inflation and deflation phases. These delays account for the non-uniform response times of the pneumatic system, including air flow rate, valve switching time, and material deformation of the Kreslings.

This coordinated actuation strategy transforms discrete actuator commands into a continuous crawling gait, enabling directional movement of the soft robotic system.

D. Wireless Communication

Wireless communication was integrated into the system to improve mobility and eliminate the need for a direct wired connection between the crawler and the control interface. This was implemented using micro-ROS running on the ESP32, allowing the embedded system to interface with a Robot Operating System (ROS 2) environment over Wi-Fi.

A micro-ROS agent was deployed on a host machine, which communicates with the ESP32 over a UDP-based Wi-Fi connection. The ESP32 operates as a subscriber node, receiving control commands published from the ROS 2 environment.

The communication architecture is structured around a shared topic, which multiple logical command types are transmitted. Two primary command modes were implemented:

- **Discrete actuator control:** Integer-based commands (1-4), corresponding to specific actions such as inflating or deflating individual Kreslings.
- **Crawling control:** Commands specifying the number of steps to execute in a crawling sequence.

Within the embedded software, these incoming messages are parsed and mapped to the corresponding control logic for actuator operation. This includes triggering specific MOSFET states for pumps and valves or initiating a multi-step crawling routine.

Initial validation of the wireless system was performed using a simple test case in which published messages toggled an onboard LED. This confirmed successful communication between the ROS2 agent and the ESP32. The same communication framework was then extended toward controlling the pneumatic actuation system.

V. Experimental Setup

The experimental evaluation of the embedded control system was conducted in a staged and methodical manner, beginning with individual component validation, and progressing toward full system integration and crawler motion testing. This incremental approach enables isolation of hardware and control issues before combining subsystems.

A. Initial Component Testing

The first phase of testing focused on validating the operation of individual hardware components, including the air pump, solenoid valve, and MOSFET divers. Each component was interfaced with the ESP32 MCU while designated GPO pins were tested using simple user-commanded control logic written in C++ with the Arduino IDE. The initial setup is shown in Fig. 5, which illustrates the physical implementation of the air pump testing configuration.

Commented [DK6]: You can focus less on demonstrating test protocol and more on your results + how you interpret those findings.

Unlike the system schematics, represented in Fig. 4, Fig. 5 highlights the physical wiring layout and component integration on the breadboard, providing insight into practical implementation details such as power distribution and real-world connections.

In this stage:

- The air pump was tested for controlled activation and deactivation through GPIO signals.

- MOSFET drivers were verified to correctly switch on so they could transfer current to the respective components.
- Power sources were used to power the hardware components due to current limitations.

These tests confirmed that the ESP32 could reliably control external pneumatic components through MOSFET-based switching. However, initial attempts at deflation revealed that the air pump alone was insufficient to remove air from the Kresling itself, leading to the incorporation of a dedicated valve-based deflation mechanism.

Fig. 5: Setup for testing the air pump and MOSFET driver.

Commented [DK7]: What does this photo show the reader that your previous diagrams didn't?

B. Subsystem Integration and Debugging

Following component-level validation, the system was expanded to include a full pneumatic loop connected to a single Kresling actuator. At this stage, both inflation (via pump) and deflation (via solenoid valves) were tested under user-controlled input. A picture showing the full hardware setup for one Kresling is in Fig. 6.

Key debugging and design iterations included:

- Refining MOSFET driver wiring to ensure stable switching behavior.
- Identifying and resolving issues related to incomplete deflation and airflow inefficiencies.

Once the Kresling demonstrated consistent and reliable inflation and deflation in response to the control inputs, without any timing errors or switching instability, the system was scaled to

include a second Kresling module. This validated the system's ability to achieve repeatable actuation. Scaling required duplicating the pump-valve-MOSFET architecture and ensuring sufficient power delivery to support simultaneous operation.

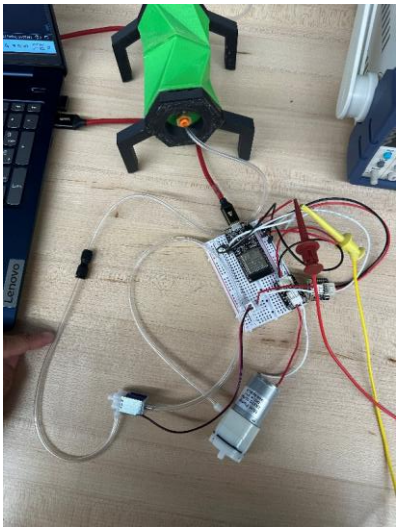


Fig.6: Full setup for one Kresling.

C. Full System Testing and Locomotion Control

With both Kreslings integrated, testing shifted toward coordinated control and crawling motion, Control sequences were implemented in embedded C++ to actuate each Kresling in a timed pattern, enabling deformation-based locomotion.

Testing revealed several system-level constraints:

- Asynchronous deflation rates between the two Kreslings required timing adjustments.
- Inability to actuate both pumps simultaneously due to power limitations.
- Mechanical inconsistencies, including leakage and structural defects in Kresling.

To address these issues, iterative testing was conducted to refine control logic and hardware performance. Actuation timing parameters were systematically adjusted to compensate for asynchronous deflation rates between the two Kreslings, ensuring more coordinated motion. Mechanical inconsistencies such as air leakage were addressed through improved sealing and inspection of the Kresling structures. These refinements resulted in more reliable and repeatable crawling behavior across multiple test cycles.

D. Wireless Communication Initial Testing (micro-ROS)

In parallel with hardware testing, wireless communication capabilities were explored using micro-ROS over Wi-Fi. Through hardware testing, ideas of how to initiate wireless communication were made. This communication was done in stages, with the first phase done with LED toggling on the ESP32. The micro-ROS code was configured such that a user publishes data to a ROS2 topic from a host machine and the ESP32 – which acts as a micro-ROS node – subscribers to that topic.

The ESP32 was configured as a subscriber node that listens to incoming messages over Wi-Fi via a micro-ROS agent running on a host machine. When a message is received, the subscriber callback processes the data and controls the LED on the microcontroller. Specifically, receiving a value of 1 turns the LED on, while 0 turns it off. Additionally, the ESP32 was created to act as a publisher by sending the status messages back to another ROS2 topic, allowing the system to confirm the LED state. An example of an RQT graph is shown in Fig. 7. An RQT graph helps visualize the ROS2 communication structure by displaying nodes and the topics connecting them, allowing for verification of correct message flow between the host system and the ESP32.

Commented [DK8]: PF: Use RQT graph in ROS to illustrate these nodes. Should take less than 5 minutes, let me know if you need help!

Initial validation included:

- Publishing binary control signals (0/1) to toggle onboard LED on MCU.
- Verifying communication through ROS2 topics and message echoing.
- Debugging network connectivity issues related to SSID configurations and host-device communication.

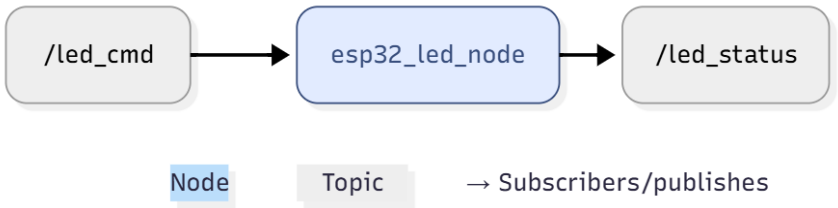


Fig. 7: Example RQT Graph of LED micro-ROS code.

E. Final Wireless Communication Testing

Once the initial testing of micro-ROS over Wi-Fi was completed, the next stage was to transition the wired communication C++ code to micro-ROS format. This was done by thinking about methods on how the user could send commands over Wi-Fi with a micro-ROS agent.

The initial method that was implemented was an 8-bit packed protocol that allows users to send commands for all commands of: inflating Kresling 1 and 2, deflating Kresling 1 and 2, and crawling commands. This would allow the user to send any of the 5 commands through an 8-bit

protocol. This protocol consisted of having the upper 4 bits correlate to the command and the lower 4 bits for the steps. This code was fully implemented and tested on the system, however, to ensure that one did not need to sit down and come up with the bits for what command they wanted, this concept was changed to a two-subscriber method.

This dual subscriber method was implemented to separate crawling control from individual Kresling testing. It allows users to execute step-based crawling sequences while independently controlling each Kresling for verification and debugging. This improves modularity and makes it easier to isolate hardware issues such as inflation/deflation failures or timing inconsistencies during testing.

1.) Initial Protocol Design: 8-bit Command Encoding

The first approach to wireless control utilized a compact 8-bit command protocol to encode both actuator commands and step parameters within a single message. This protocol was designed to minimize communication overhead while enabling flexible control of the system.

The structure of the 8-bit message was defined as follows:

- **Upper 4 bits:** Command indicator
- **Lower 4 bits:** Number of Crawling steps

Using this encoding, a single byte could represent multiple control actions: inflation of Kresling 1 or 2, deflation of Kresling 1 or 2, and execution of crawling sequence with specified number of steps. This approach was successfully implemented and tested, demonstrating that compact, low-bandwidth communication could effectively control the embedded system.

However, despite this design working, it introduced usability and stability issues. Some issues were that users had to manually construct binary command values, which would increase the likelihood of input errors. Another was, that encoding scheme reduced readability and would make debugging a bit challenging. Lastly, extending the protocol to support additional features would require redefining the bit structure. These limitations made the system less intuitive for real-time interaction, particularly in a development and testing environment.

2.) Final Implementation: Dual-Subscriber Architecture

To address these challenges, the communication architecture was redesigned to a dual-subscriber model within micro-ROS. In this approach, separate message streams were used for different control functions, improving clarity and modularity.

The final system utilized:

- **Subscriber 1:** Receives integer-based commands corresponding to discrete actuator actions (e.g., inflate/deflate specific Kreslings)

- **Subscriber 2:** Receives integer values representing the number of steps for the crawling sequence

Both subscribers listen to a shared communication framework but processed incoming data independently within the embedded software. This separation of commands simplifies command interpretation and reduces ambiguity in control logic. This would allow users to run crawling sequence while simultaneously being able to run tests on inflating/deflating the Kresling. An image showing the RQT graph of the subscribers is in Fig. 8.

During this testing, the actuation timings for the crawling sequence were iteratively fine-tuned. It was observed that one Kresling required a longer duration to fully deflate, which was due to a tighter construction of it. To address this, the control logic was modified to dynamically adjust actuation timing across iterations. Specifically, longer actuation durations were used during the initial steps, followed by a gradual decrease in timing for subsequent steps. This approach improved synchronization between the two actuators and results in more consistent crawling behavior.

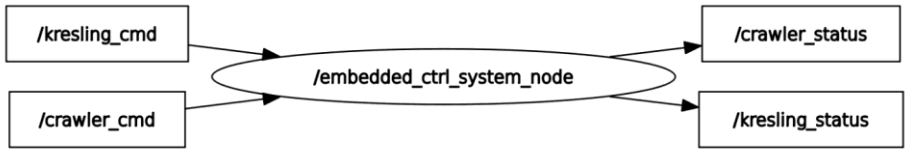


Fig. 8: RQT graph of two-subscriber code

VI. Results

The implemented system successfully demonstrates a functional, wireless embedded control system for a soft robotic Crawler. The ESP32 microcontroller, combined with the MOSFETs and pneumatic components, enabled reliable control for air flow for both Kreslings. Through the micro-ROS communication framework, users can send wireless commands to execute both individual actuator control and multi-step crawling sequences. All source code and demonstration videos of the system are available in the accompanying GitHub repository, which provides supporting documentation of the implemented control system and experimental results.

System performance was evaluated using qualitative metrics focused on functional validation and system integration. Experimental testing confirmed actuation reliability, as each Kresling consistently inflated and deflated in response to user commands. The system also exhibited proper response time, with minimal observable delay between command input and actuator activation. Additionally, successful execution of multi-step crawling sequences demonstrated effective synchronization between the two Kresling modules. The system-maintained power

stability during operation, with no observable voltage drops or component failures when both actuators were coordinated crawling sequence. Finally mechanical performance was validated through visible and repeatable crawling motion resulting from coordinated deformation.

However, the overall motion of the Crawlers' performance was limited to various factors. The Crawler exhibited inconsistent forward movement due to insufficient friction between the Kresling structures and the ground surface. This resulted in the Crawler staying in place during stepping sequences, which reduced the effectiveness of the crawling motion. Furthermore, variations in deflation timing between the two Kreslings led to imperfect synchronization, which impacted the smoothness of motion.

Overall, the results demonstrate that while the system is capable of controlled actuation and basic crawling behavior, improvements in the mechanical design are necessary to achieve more reliable and efficient motion.

VII. Improvements & Future Work

While the developed system successfully demonstrates embedded control of a soft robotic crawler, several limitations can be identified that present opportunities for future improvement across mechanical design, electrical integration, power management, and control strategy. One key area for enhancement is the mechanical design of the crawler. The current system relies solely on deformation of the Kresling actuators to generate motion, which limits control over direction and overall movement. Incorporating a dedicated leg design or friction-enhancing contact structure would enable a more controlled and repeatable walking style. This could include the use of friction pads or bio-inspired legs, allowing for improved interaction between the soft body and the ground surface.

Another important improvement involves the electrical design and system integration. The current implementation relies on a breadboard-based wiring setup with numerous external connections, which introduces reliability issues during operation. Wires can get loose or disconnected, particularly during repeated actuation cycles. Developing a custom breakout board or printed circuit board (PCB) to integrate the ESP32, MOSFET drivers, and actuator connections would significantly reduce wiring complexity, improve robustness, and enhance the overall stability of the system.

Power management also remains a limiting factor in the current design. The crawler is powered through an external power supply, which restricts its mobility and prevents fully untethered operation. Additionally, testing revealed that simultaneous actuation of multiple pumps can lead to reduced performance, likely due to insufficient current delivery. Future iterations of the system would benefit from the integration of a dedicated onboard power source, such as a battery pack with appropriate voltage regulation, to support higher current demands and enable fully autonomous operation.

In terms of control and communication, the current system operates primarily in an open-loop manner and lacks feedback mechanisms for precise actuation. Incorporating sensors, such as pressure or flex sensors, would enable closed-loop control and improve the accuracy and repeatability of the crawling motion. Furthermore, refining the wireless communication framework, such as implementing a more unified and scalable command structure within the micro-ROS architecture, would enhance system usability and flexibility.

Overall, these improvements would transition the current prototype from a proof-of-concept system into a more robust, scalable, and fully autonomous soft robotic platform, capable of more efficient and reliable locomotion.

VIII. Conclusion

This project successfully demonstrated the design and implementation of an embedded control system for a soft robotic crawler actuated by Kresling modules. By integrating an ESP32 microcontroller with MOSFET drivers, air pumps, and solenoid valves, the system was able to achieve controlled inflation and deflation of two independent actuators. The development process followed a staged and iterative approach, beginning with individual component validation and progressing toward full system integration and coordinated crawling behavior.

A key outcome of this project was the successful implementation of wireless communication using micro-ROS over Wi-Fi, allowing remote command execution for both individual actuator control and multi-step crawling sequences. This highlights the feasibility of combining embedded systems with modern communication frameworks to enhance the flexibility and usability of soft robotic platforms.

Experimental results demonstrated that the Crawler could produce observable motion through timed actuation sequences. However, the system also revealed important challenges, including limitations in power delivery, actuator synchronization, and mechanical interaction with the environment. These findings highlight the complexity of controlling soft robotic systems, particularly when operating under open-loop conditions.

Overall, this project provides a functional proof-of-concept for embedded control of soft robotic locomotion and established a foundation for future work in improving mechanical design, control strategies, and system integration. The insights gained contribute to the broader field of soft robotics, where adaptable, compliant systems have strong potential for applications in constrained environments such as medical devices and inspection tasks.

Bibliography

[1] S. I. Rich, R. J. Wood, and C. Majidi, “Untethered soft robotics,” *Nature Electronics*, vol. 1, no. 2, pp. 102–112, Feb. 2018, doi: https://doi.org/10.1038/s41928-018-0024-1.

[2] C. E. Staff, “Know when to use open- or closed-loop control - Control Engineering,” *Control Engineering*, Aug. 17, 2017. https://www.controleng.com/know-when-to-use-open-or-closed-loop-control/

[3] S. Kline, “Open Loop vs Closed Loop Systems in Robotics: Key Differences and Applications,” *techiny.com*, 2025. https://techiny.com/robotics/open-loop-vs-closed-loop?utm_source=chatgpt.com (accessed Apr. 21, 2026).

[4] E. Budny and M. Chlosta, “An Open-Loop vs. Closed-Loop Backhoe Excavator Control System,” *Proceedings of the International Symposium on Automation and Robotics in Construction (IAARC)*, Oct. 2006, doi: https://doi.org/10.22260/isarc2006/0105.

[5] Q. Ze *et al.*, “Soft robotic origami crawler,” *Science Advances*, vol. 8, no. 13, Apr. 2022, doi: https://doi.org/10.1126/sciadv.abm7834.

[6] W. Xue, B. Jian, L. Jin, R. Wang, and Q. Ge, “Origami Robots: Design, Actuation, and 3D Printing Methods,” *Advanced Materials Technologies*, May 2025, doi: https://doi.org/10.1002/admt.202500278.

[7] B. Becher, “Soft Robotics, Explained | Built In,” *Built In*, 2024. https://builtin.com/articles/soft-robotics

[8] GeeksforGeeks, “Difference between OpenLoop Control System and ClosedLoop Control System,” *GeeksforGeeks*, Jan. 06, 2020. https://www.geeksforgeeks.org/electronics-engineering/difference-between-open-loop-control-system-and-closed-loop-control-system/